

Review Questions

- 5.1. List four design issues for which the concept of concurrency is relevant.
- 5.2. What are three contexts in which concurrency arises?
- 5.3. What is the basic requirement for the execution of concurrent processes?
- 5.4. List three degrees of awareness between processes and briefly define each.
- 5.5. What is the distinction between competing processes and cooperating processes?
- 5.6. List the three control problems associated with competing processes and briefly define each.
- 5.7. List the requirements for mutual exclusion.
- 5.8. What operations can be performed on a semaphore?
- 5.9. What is the difference between binary and general semaphores?
- 5.10. What is the difference between strong and weak semaphores?
- 5.11. What is a monitor?
- 5.12. What is the distinction between *blocking* and *nonblocking* with respect to messages?
- 5.13. What conditions are generally associated with the readers/writers problem?

Problems

- 5.1. At the beginning of Section 5.1, it is stated that multiprogramming and multiprocessing present the same problems, with respect to concurrency. This is true as far as it goes. However, cite two differences in terms of concurrency between multiprogramming and multiprocessing.
- 5.2. Processes and threads provide a powerful structuring tool for implementing programs that would be much more complex as simple sequential programs. An earlier construct that is instructive to examine is the coroutine. The purpose of this problem is to introduce coroutines and compare them to processes. Consider this simple problem from [CONW63]:

Read 80-column cards and print them on 125-character lines, with the following changes. After every card image an extra blank is inserted, and every adjacent pair of asterisks (**) on a card is replaced by the character.

- a. Develop a solution to this problem as an ordinary sequential program. You will find that the program is tricky to write. The interactions among the various elements of the program are uneven because of the conversion from a length of 80 to 125; furthermore, the length of the card image, after conversion, will vary depending on the number of double asterisk occurrences. One way to improve clarity, and to minimize the potential for bugs, is to write the application as three separate procedures. The first procedure reads in card images, pads each image with a blank, and writes a stream of characters to a temporary file. After all of the cards have been read, the second procedure reads the temporary file, does the character substitution, and writes out a second temporary file. The third procedure reads the stream of characters from the second temporary file and prints lines of 125 characters each.
- b. The sequential solution is unattractive because of the overhead of I/O and temporary files. Conway proposed a new form of program structure, the coroutine, that allows the application to be written as three programs connected by one-character buffers (Figure 5.25). In a traditional **procedure**, there is a master/slave relationship between the called and calling procedure. The calling procedure may execute a call from any point in the procedure; the called procedure is begun at its entry point and returns to the calling procedure at the point of call. The **coroutine** exhibits a more symmetric relationship. As each call is made, execution takes up from the

```

char rs, sp;
char inbuf[80], outbuf[125];
void read()
{
    while (true) {
        READCARD (inbuf);
        for (int i=0; i < 80; i++){
            rs = inbuf [i];
            RESUME squash
        }
        rs = " ";
        RESUME squash;
    }
}
void print()
{
    while (true) {
        for (int j = 0; j < 125; j++){
            outbuf [j] = sp;
            RESUME squash
        }
        OUTPUT (outbuf);
    }
}

void squash()
{
    while (true) {
        if (rs != "**") {
            sp = rs;
            RESUME print;
        }
        else{
            RESUME read;
            if (rs == "**") {
                sp = " ";
                RESUME print;
            }
        }
        else {
            sp = "**";
            RESUME print;
            sp = rs;
            RESUME print;
        }
    }
    RESUME read;
}

```



Figure 5.25 An Application of Coroutines

last active point in the called procedure. Because there is no sense in which a calling procedure is “higher” than the called, there is no return. Rather, any coroutine can pass control to any other coroutine with a resume command. The first time a coroutine is invoked, it is “resumed” at its entry point. Subsequently, the coroutine is reactivated at the point of its own last resume command. Note that only one coroutine in a program can be in execution at one time and that the transition points are explicitly defined in the code, so this is not an example of concurrent processing. Explain the operation of the program in Figure 5.25.

- c. The program does not address the termination condition. Assume that the I/O routine READCARD returns the value true if it has placed an 80-character image in *inbuf*; otherwise it returns false. Modify the program to include this contingency. Note that the last printed line may therefore contain less than 125 characters.
- d. Rewrite the solution as a set of three processes using semaphores.

5.3. Consider the following program:

```

P1: {
    shared int x;
    x = 10;
    while (1) {
        x = x - 1;
        x = x + 1;
        if (x != 10)
            printf("x is %d", x)
        }
    }
}

P2: {
    shared int x;
    x = 10;
    while ( 1 ) {
        x = x - 1;
        x = x + 1;
        if (x!=10)
            printf("x is %d", x)
        }
    }
}

```

Note that the scheduler in a uniprocessor system would implement pseudo parallel execution of these two concurrent processes by interleaving their instructions, without restriction on the order of the interleaving.

- a. Show a sequence (i.e., trace the sequence of interleavings of statements) such that the statement “x is 10” is printed.
- b. Show a sequence such that the statement “x is 8” is printed. You should remember that the increment/decrements at the source language level are not done atomically, that is, the assembly language code:

```
LD    R0,X /* load R0 from memory location x */
INCR  R0    /* increment R0 */
STO   R0,X /* store the incremented value back in X */
```

implements the single C increment instruction ($x = x + 1$).

- 5.4. Consider the following program:

```
const int n = 50;
int tally;
void total()
{
    int count;
    for (count = 1; count <= n; count++) {
        tally++;
    }
}
void main()
{
    tally = 0;
    parbegin (total (), total ());
    write (tally);
}
```

- a. Determine the proper lower bound and upper bound on the final value of the shared variable *tally* output by this concurrent program. Assume processes can execute at any relative speed and that a value can only be incremented after it has been loaded into a register by a separate machine instruction.
 - b. Suppose that an arbitrary number of these processes are permitted to execute in parallel under the assumptions of part (a). What effect will this modification have on the range of final values of *tally*?
- 5.5. Is busy waiting always less efficient (in terms of using processor time) than a blocking wait? Explain.
 - 5.6. Consider the following program:

```
boolean blocked [2];
int turn;
void P (int id)
{
    while (true) {
        blocked[id] = true;
        while (turn != id) {
            while (blocked[1-id])
                /* do nothing */;
            turn = id;
        }
        /* critical section */
        blocked[id] = false;
        /* remainder */
    }
}
```

```

void main()
{
    blocked[0] = false;
    blocked[1] = false;
    turn = 0;
    parbegin (P(0), P(1));
}

```

This software solution to the mutual exclusion problem for two processes is proposed in [HYMA66]. Find a counterexample that demonstrates that this solution is incorrect. It is interesting to note that even the *Communications of the ACM* was fooled on this one.

- 5.7.** A software approach to mutual exclusion is Lamport's **bakery algorithm** [LAMP74], so called because it is based on the practice in bakeries and other shops in which every customer receives a numbered ticket on arrival, allowing each to be served in turn. The algorithm is as follows:

```

boolean choosing[n];
int number[n];
while (true) {
    choosing[i] = true;
    number[i] = 1 + getmax(number[], n);
    choosing[i] = false;
    for (int j = 0; j < n; j++) {
        while (choosing[j]) { };
        while ((number[j] != 0) && (number[j], j) < (number[i], i)) { };
    }
    /* critical section */;
    number[i] = 0;
    /* remainder */;
}

```

The arrays *choosing* and *number* are initialized to false and 0, respectively. The *i*th element of each array may be read and written by process *i* but only read by other processes. The notation $(a, b) < (c, d)$ is defined as:

$$(a < c) \text{ or } (a = c \text{ and } b < d)$$

- a.** Describe the algorithm in words.
 - b.** Show that this algorithm avoids deadlock.
 - c.** Show that it enforces mutual exclusion.
- 5.8.** Now consider a version of the bakery algorithm without the variable *choosing*. Then we have

```

1 int number[n];
2 while (true) {
3     number[i] = 1 + getmax(number[], n);
4     for (int j = 0; j < n; j++) {
5         while ((number[j] != 0) && (number[j], j) < (number[i], i)) { };
6     }
7     /* critical section */;
8     number[i] = 0;
9     /* remainder */;
10 }

```

Does this version violate mutual exclusion? Explain why or why not.

- 5.9.** Consider the following program which provides a software approach to mutual exclusion:

integer array control [1 :N]; **integer** k
 where $1 \leq k \leq N$, and each element of “control” is either 0, 1,
 or 2. All elements of “control” are initially zero; the initial value
 of k is immaterial.

The program of the *i*th process ($1 \leq i \leq N$) is

```

begin integer j;
L0: control [i] := 1;
L1: for j:=k step 1 until N, 1 step 1 until k do
    begin
        if j = i then goto L2;
        if control [j] ≠ 0 then goto L1
    end;
L2: control [i] := 2;
    for j := 1 step 1 until N do
        if j ≠ i and control [j] = 2 then goto L0;
L3: if control [k] ≠ 0 and k ≠ i then goto L0;
L4: k := i;
    critical section;
L5: for j := k step 1 until N, 1 step 1 until k do
    if j ≠ k and control [j] ≠ 0 then
        begin
            k := j;
            goto L6
        end;
L6: control [i] := 0;
L7: remainder of cycle;
    goto L0;
end

```

This is referred to as the Eisenberg-McGuire algorithm. Explain its operation and its key features.

- 5.10.** Consider the first instance of the statement `bolt = 0` in Figure 5.2b.
- Achieve the same result using the exchange instruction.
 - Which method is preferable?
- 5.11.** When a special machine instruction is used to provide mutual exclusion in the fashion of Figure 5.2, there is no control over how long a process must wait before being granted access to its critical section. Devise an algorithm that uses the `compare&swap` instruction but that guarantees that any process waiting to enter its critical section will do so within $n - 1$ turns, where n is the number of processes that may require access to the critical section and a “turn” is an event consisting of one process leaving the critical section and another process being granted access.
- 5.12.** Consider the following definition of semaphores:

```

void semWait(s)
{
    if (s.count > 0) {
        s.count--;
    }
    else {
        place this process in s.queue;
        block;
    }
}

```


The solution appears to do everything right: All accesses to the shared variables are protected by mutual exclusion, processes do not block themselves while in the mutual exclusion, new processes are prevented from using the resource if there are (or were) three active users, and the last process to depart unblocks up to three waiting processes.

- a. The program is nevertheless incorrect. Explain why.
- b. Suppose we change the if in line 6 to a while. Does this solve any problem in the program? Do any difficulties remain?

5.14. Now consider this correct solution to the preceding problem:

```

1  semaphore mutex = 1, block = 0;           /* share variables: semaphores, */
2  int active = 0, waiting = 0;               /* counters, and */
3  boolean must_wait = false;                /* state information */
4
5  semWait(mutex);                           /* Enter the mutual exclusion */
6  if(must_wait) {                           /* If there are (or were) 3, then */
7      ++waiting;                             /* we must wait, but we must leave */
8      semSignal(mutex);                     /* the mutual exclusion first */
9      semWait(block);                       /* Wait for all current users to depart */
10 } else {
11     ++active;                             /* Update active count, and */
12     must_wait = active == 3;              /* remember if the count reached 3 */
13     semSignal(mutex);                     /* Leave mutual exclusion */
14 }
15
16 /* critical section */
17
18 semWait(mutex);                           /* Enter mutual exclusion */
19 --active;                                 /* and update the active count */
20 if(active == 0) {                         /* Last one to leave? */
21     int n;
22     if (waiting < 3) n = waiting;
23     else n = 3;                           /* If so, see how many processes to unblock */
24     waiting -= n;                         /* Deduct this number from waiting count */
25     active = n;                           /* and set active to this number */
26     while( n > 0 ) {                       /* Now unblock the processes */
27         semSignal(block);                 /* one by one */
28         --n;
29     }
30     must_wait = active == 3;              /* Remember if the count is 3 */
31 }
32 semSignal(mutex);                         /* Leave the mutual exclusion */

```

- a. Explain how this program works and why it is correct.
- b. This solution does not completely prevent newly arriving processes from cutting in line but it does make it less likely. Give an example of cutting in line.
- c. This program is an example of a general design pattern that is a uniform way to implement solutions to many concurrency problems using semaphores. It has been referred to as the **I'll Do It For You** pattern. Describe the pattern.

5.15. Now consider another correct solution to the preceding problem:

```

1  semaphore mutex = 1, block = 0;           /* share variables: semaphores, */
2  int active = 0, waiting = 0;               /* counters, and */
3  boolean must_wait = false;                /* state information */
4

```

```

5  semWait(mutex);                                /* Enter the mutual exclusion */
6  if(must_wait) {                                /* If there are (or were) 3, then */
7      ++waiting;                                /* we must wait, but we must leave */
8      semSignal(mutex);                          /* the mutual exclusion first */
9      semWait(block);                            /* Wait for all current users to depart */
10     --waiting;                                /* We've got the mutual exclusion; update count */
11 }
12 ++active;                                       /* Update active count, and remember */
13 must_wait = active == 3;                       /* if the count reached 3 */
14 if(waiting > 0 && !must_wait)                  /* If there are others waiting */
15     semSignal(block);                          /* and we don't yet have 3 active, */
16                                             /* unblock a waiting process */
17 else semSignal(mutex);                        /* otherwise open the mutual exclusion */
18
19 /* critical section */
20
21 semWait(mutex);                                /* Enter mutual exclusion */
22 --active;                                       /* and update the active count */
23 if(active == 0)                                /* If last one to leave? */
24     must_wait = false;                         /* set up to let new processes enter */
25 if(waiting == 0 && !must_wait)                  /* If there are others waiting */
26     semSignal(block);                          /* and we don't have 3 active, */
27                                             /* unblock a waiting process */
28 else semSignal(mutex);                        /* otherwise open the mutual exclusion */

```

- a. Explain how this program works and why it is correct.
- b. Does this solution differ from the preceding one in terms of the number of processes that can be unblocked at a time? Explain.
- c. This program is an example of a general design pattern that is a uniform way to implement solutions to many concurrency problems using semaphores. It has been referred to as the **Pass The Baton** pattern. Describe the pattern.

5.16. It should be possible to implement general semaphores using binary semaphores. We can use the operations `semWaitB` and `semSignalB` and two binary semaphores, `delay` and `mutex`. Consider the following:

```

void semWait(semaphore s)
{
    semWaitB(mutex);
    s--;
    if (s < 0) {
        semSignalB(mutex);
        semWaitB(delay);
    }
    else semSignalB(mutex);
}

void semSignal(semaphore s);
{
    semWaitB(mutex);
    s++;
    if (s <= 0)
        semSignalB(delay);
    semSignalB(mutex);
}

```

Initially, `s` is set to the desired semaphore value. Each `semWait` operation decrements `s`, and each `semSignal` operation increments `s`. The binary semaphore `mutex`, which is initialized to 1, assures that there is mutual exclusion for the updating of `s`. The binary semaphore `delay`, which is initialized to 0, is used to block processes.

There is a flaw in the preceding program. Demonstrate the flaw and propose a change that will fix it. *Hint:* Suppose two processes each call `semWait(s)` when `s` is initially 0, and after the first has just performed `semSignalB(mutex)` but not performed `semWaitB(delay)`, the second call to `semWait(s)` proceeds to the same point. All that you need to do is move a single line of the program.

- 5.17.** In 1978, Dijkstra put forward the conjecture that there was no solution to the mutual exclusion problem avoiding starvation, applicable to an unknown but finite number of processes, using a finite number of weak semaphores. In 1979, J. M. Morris refuted this conjecture by publishing an algorithm using three weak semaphores. The behavior of the algorithm can be described as follows: If one or several process are waiting in a `semWait(S)` operation and another process is executing `semSignal(S)`, the value of the semaphore `S` is not modified and one of the waiting processes is unblocked independently of `semWait(S)`. Apart from the three semaphores, the algorithm uses two nonnegative integer variables as counters of the number of processes in certain sections of the algorithm. Thus, semaphores `A` and `B` are initialized to 1, while semaphore `M` and counters `NA` and `NM` are initialized to 0. The mutual exclusion semaphore `B` protects access to the shared variable `NA`. A process attempting to enter its critical section must cross two barriers represented by semaphores `A` and `M`. Counters `NA` and `NM`, respectively, contain the number of processes ready to cross barrier `A` and those having already crossed barrier `A` but not yet barrier `M`. In the second part of the protocol, the `NM` processes blocked at `M` will enter their critical sections one by one, using a cascade technique similar to that used in the first part. Define an algorithm that conforms to this description.

- 5.18.** The following problem was once used on an exam:

Jurassic Park consists of a dinosaur museum and a park for safari riding. There are m passengers and n single-passenger cars. Passengers wander around the museum for a while, then line up to take a ride in a safari car. When a car is available, it loads the one passenger it can hold and rides around the park for a random amount of time. If the n cars are all out riding passengers around, then a passenger who wants to ride waits; if a car is ready to load but there are no waiting passengers, then the car waits. Use semaphores to synchronize the m passenger processes and the n car processes.

The following skeleton code was found on a scrap of paper on the floor of the exam room. Grade it for correctness. Ignore syntax and missing variable declarations. Remember that `P` and `V` correspond to `semWait` and `semSignal`.

```
resource Jurassic_Park()
    sem car_avail := 0, car_taken := 0, car_filled := 0, passenger_released := 0
    process passenger(i := 1 to num_passengers)
        do true -> nap(int(random(1000*wander_time)))
            P(car_avail); V(car_taken); P(car_filled)
            P(passenger_released)
        od
    end passenger

    process car(j := 1 to num_cars)
        do true -> V(car_avail); P(car_taken); V(car_filled)
            nap(int(random(1000*ride_time)))
            V(passenger_released)
        od
    end car
end Jurassic_Park
```

- 5.19.** In the commentary on Figure 5.9 and Table 5.4, it was stated that “it would not do simply to move the conditional statement inside the critical section (controlled by `s`) of the consumer because this could lead to deadlock.” Demonstrate this with a table similar to Table 5.4.

5.20. Consider the solution to the infinite-buffer producer/consumer problem defined in Figure 5.10. Suppose we have the (common) case in which the producer and consumer are running at roughly the same speed. The scenario could be:

```
Producer: append; semSignal; produce; ... ; append; semSignal; produce; ...  
Consumer: consume; ... ; take; semWait; consume; ... ; take; semWait; ...
```

The producer always manages to append a new element to the buffer and signal during the consumption of the previous element by the consumer. The producer is always appending to an empty buffer and the consumer is always taking the sole item in the buffer. Although the consumer never blocks on the semaphore, a large number of calls to the semaphore mechanism is made, creating considerable overhead.

Construct a new program that will be more efficient under these circumstances. *Hints:* Allow *n* to have the value -1, which is to mean that not only is the buffer empty but that the consumer has detected this fact and is going to block until the producer supplies fresh data. The solution does not require the use of the local variable *m* found in Figure 5.10.

- 5.21.** Consider Figure 5.13. Would the meaning of the program change if the following were interchanged?
- a.** semWait(e) ; semWait(s)
 - b.** semSignal(s) ; semSignal(n)
 - c.** semWait(n) ; semWait(s)
 - d.** semSignal(s) ; semSignal(e)
- 5.22.** The following pseudocode is a correct implementation of the producer/consumer problem with a bounded buffer:

<pre>item[3] buffer; // initially empty semaphore empty; // initialized to +3 semaphore full; // initialized to 0 binary_semaphore mutex; // initialized to 1</pre>	
<pre>void producer() { ... while (true) { item = produce(); p1: wait(empty); / wait(mutex); p2: append(item); \ signal(mutex); p3: signal(full); } }</pre>	<pre>void consumer() { ... while (true) { c1: wait(full); / wait(mutex); c2: item = take(); \ signal(mutex); c3: signal(empty); consume(item); } }</pre>

Labels p1, p2, p3 and c1, c2, c3 refer to the lines of code shown above (p2 and c2 each cover three lines of code). Semaphores empty and full are linear semaphores that can take unbounded negative and positive values. There are multiple producer processes, referred to as Pa, Pb, Pc, etc., and multiple consumer processes, referred to as Ca, Cb, Cc, etc. Each semaphore maintains a FIFO (first-in-first-out) queue of blocked processes. In the scheduling chart below, each line represents the state of the buffer and semaphores after the scheduled execution has occurred. To simplify, we assume that scheduling is such that processes are never interrupted while executing a given portion of code p1, or p2, ..., or c3. Your task is to complete the following chart.

Scheduled Step of Execution	full's State and Queue	Buffer	empty's State and Queue
Initialization	full = 0	OOO	empty = +3
Ca executes c1	full = -1 (Ca)	OOO	empty = +3
Cb executes c1	full = -2 (Ca, Cb)	OOO	empty = +3
Pa executes p1	full = -2 (Ca, Cb)	OOO	empty = +2
Pa executes p2	full = -2 (Ca, Cb)	X OO	empty = +2
Pa executes p3	full = -1 (Cb) Ca	X OO	empty = +2
Ca executes c2	full = -1 (Cb)	OOO	empty = +2
Ca executes c3	full = -1 (Cb)	OOO	empty = +3
Pb executes p1	full =		empty =
Pa executes p1	full =		empty =
Pa executes __	full =		empty =
Pb executes __	full =		empty =
Pb executes __	full =		empty =
Pc executes p1	full =		empty =
Cb executes __	full =		empty =
Pc executes __	full =		empty =
Cb executes __	full =		empty =
Pa executes __	full =		empty =
Pb executes p1-p3	full =		empty =
Pc executes __	full =		empty =
Pa executes p1	full =		empty =
Pd executes p1	full =		empty =
Ca executes c1-c3	full =		empty =
Pa executes __	full =		empty =
Cc executes c1-c2	full =		empty =
Pa executes __	full =		empty =
Cc executes c3	full =		empty =
Pd executes p2-p3	full =		empty =

- 5.23.** This problem demonstrates the use of semaphores to coordinate three types of processes.⁶ Santa Claus sleeps in his shop at the North Pole and can only be awakened by either (1) all nine reindeer being back from their vacation in the South Pacific, or (2) some of the elves having difficulties making toys; to allow Santa to get some sleep, the elves can only wake him when three of them have problems. When three elves are having their problems solved, any other elves wishing to visit Santa must wait for those elves to return. If Santa wakes up to find three elves waiting at his shop's door, along with the last reindeer having come back from the tropics, Santa has decided that the elves can wait until after Christmas, because it is more important to get his sleigh ready. (It is assumed that the reindeer do not want to leave the tropics, and therefore they stay there until the last possible moment.) The last reindeer to arrive must get Santa while the others wait in a warming hut before being harnessed to the sleigh. Solve this problem using semaphores.

⁶I am grateful to John Trono of St. Michael's College in Vermont for supplying this problem.

- 5.24.** Show that message passing and semaphores have equivalent functionality by
- Implementing message passing using semaphores. *Hint:* Make use of a shared buffer area to hold mailboxes, each one consisting of an array of message slots.
 - Implementing a semaphore using message passing. *Hint:* Introduce a separate synchronization process.
- 5.25.** Explain what is the problem with this implementation of the one-writer many-readers problem?

```

int readcount;                // shared and initialized to 0
Semaphore mutex, wrt;         // shared and initialized to 1;

// Writer :
semWait(wrt);
/* Writing performed*/
semSignal(wrt);

// Readers :
semWait(mutex);
readcount := readcount + 1;
if readcount == 1 then semWait(wrt);
semSignal(mutex);
/*reading performed*/
semWait(mutex);
readcount := readcount - 1;
if readcount == 0 then Up(wrt);
semSignal(mutex);

```